



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

(SECP38843) SPECIAL TOPIC IN DATA ENGINEERING

SECTION 02

GROUP 3

TUTORIAL 4: GAI

THE AI WORKFLOW FOR DATA ENGINEERING

LECTURER: DR. ARYATI BINTI BAKRI

STUDENT NAME	MATRIC NO
GUI KAH SIN	A23CS0080
SABRINA HENG WEI QI	A23CS0265
WOO CHENG SHUAN	A23CS0283

Table of Contents

1.0	What is AI-assisted data engineering	3
1.1	Data Engineering.....	3
1.2	AI-Assisted Data Engineering	3
2.0	What is Claude/AI agent use in the tutorial?	4
2.1	What is Claude?	4
2.2	What is an AI Agent?	5
2.3	Role of Claude in This Tutorial	5
3.0	Overview of the Data Pipeline.....	7
4.0	Pipeline Development and Implementation	8
4.1	Environment Setup and Verification	9
4.2	Exploration of Source Data	10
4.3	AI-Assisted Pipeline Development Using Claude	13
4.4	Project Structure and Model Development.....	15
4.4.1	Staging Layer Development	16
4.4.2	Intermediate Layer Development	18
4.4.3	Mart Layer Development	19
4.4.4	Data Quality and Documentation Configuration	19
4.5	Pipeline Execution, Testing and Documentation	21
4.6	Results in Snowflake.....	29
4.7	Error Detection and Correction	30
5.0	Use Case Diagram.....	32
6.0	Reflection	33
7.0	Reference.....	35

1.0 What is AI-assisted data engineering

The increasing volume and complexity of data generated nowadays have created a growing demand for efficient data engineering solutions. Previously, data engineers are responsible for building and maintaining the data pipelines from ingest to transform and prepare data for analytical purposes. However, the emergence of Artificial Intelligence (AI) has introduced new opportunities to automate many of these development tasks. This section introduces the fundamental concepts of data engineering and explains how AI technologies can be integrated into the data engineering workflow to improve efficiency, productivity and development speed.

1.1 Data Engineering

Data engineering is the discipline responsible for designing, building and maintaining systems that collect, store and process data for analytical purposes (Sikka et al., 2023). Data engineers develop data pipelines that ingest data from source systems to storage platforms such as data warehouses, where the data can be analyzed and used for decision-making.

Conventionally, data engineers perform tasks such as designing SQL queries, setting up data sources, creating transformation logic, conducting data quality testing and preparing documentation. These tasks ensure that raw data is transformed into a structured and reliable format that is analysis-ready. However, performing these tasks manually can be time-consuming, especially when dealing with large and complex datasets.

1.2 AI-Assisted Data Engineering

AI-assisted data engineering refers to the use of artificial intelligence (AI) technologies to automate and support various tasks involved in building and managing data pipelines. Instead of manually creating every component of the pipeline, engineers can provide instructions in natural language and allow AI systems to generate the required code, configurations and documentation.

Many tasks that previously required manual effort can now be executed efficiently using AI-assisted tools. These tools can automatically generate the required code, configure data sources, create transformation logic, perform documentation tasks and assist with

debugging activities by interpreting the user's needs. As a result, development time can be reduced while improving productivity and reducing the likelihood of human error.

In this tutorial, AI-assisted data engineering is implemented using Claude AI integrated within Cursor Pro (Cursor, 2026). Instead of manually creating every component of the pipeline, the user provides prompts describing the required tasks and the AI generates the corresponding dbt models, SQL scripts and configuration files. This allows engineers to focus more on designing the data architecture, validating outputs and ensuring that business requirements are met.

However, AI-assisted data engineering does not replace the role of data engineers. Engineers still need to review the generated code, verify data quality and make important decisions regarding data modeling and pipeline design. Therefore, AI serves as a productivity-enhancing tool that assists engineers rather than replacing their expertise.

2.0 What is Claude/AI agent use in the tutorial?

Recent advancements in Artificial Intelligence have enabled AI systems to perform tasks beyond simple question answering. These systems can assist developers by generating code, creating documentation and helping to solve technical problems. In this tutorial, Claude AI was used through Cursor Pro to support the development of the data pipeline. Therefore, it is essential to understand both Claude and the concept of AI agents before examining their role in the tutorial.

2.1 What is Claude?

Claude is a Large Language Model (LLM) developed by Anthropic that can understand natural language instructions and generate human-like text responses (Anthropic, 2024). Large Language Models are trained on large volumes of text data, enabling them to understand context, answer questions and assist with problem-solving tasks. These capabilities make Claude a useful tool for data engineering projects. Within this tutorial, Claude was integrated into Cursor Pro, an AI-powered development environment that allows users to interact with AI models directly during project development. This integration enables Claude to assist in the development of the data pipeline.

2.2 What is an AI Agent?

An AI agent is a software system that uses artificial intelligence to perceive information, make decisions and perform actions to achieve specific goals with varying degrees of autonomy (Russell & Norvig, 2021). Unlike conventional chatbots that primarily generate responses to user queries, AI agents can execute multi-step tasks, interact with tools and external systems and adapt their actions based on the context of the task.

AI agents are increasingly used because they can automate repetitive activities and improve development efficiency. By interpreting user requirements expressed in natural language, AI agents can assist with code generation, file management, testing and other development tasks, helping developers complete work more effectively.

2.3 Role of Claude in This Tutorial

In this tutorial, Claude acts as an AI agent within Cursor Pro's Agent Mode. Unlike the conventional chatbot that only provides suggestions, Claude can directly interact with the existing project files, generate the code, modify configurations as well as assist in managing the development workflow. As a result, this enables Claude to play an active role in the pipeline's development.

Claude is used throughout the tutorial to automate several important tasks. In order to prepare raw data for additional processing, it first creates source definitions and staging models. Furthermore, it creates SQL transformation scripts and YAML configuration files required for the dbt project. Through simple language prompts, the user can instruct Claude to build different layers of the data pipeline, including staging, intermediate and mart models.

Besides generating code, Claude helps configure project settings by updating dbt configuration files and appropriate materialization methods for different models. For instance, Claude helps configure model materialisation settings in Snowflake by using views for staging and intermediate models, while materialising the final mart model as a table for reporting purposes. The AI agent also assists with debugging by analyzing error messages produced during pipeline execution and suggesting or applying the corrections. Moreover, Claude will automatically generate documentation, including model

descriptions, column explanations and data quality tests, which improves the maintainability of the project.

The use of Claude significantly speeds up the development process because repetitive coding tasks are automated. Therefore, data engineers can spend more time focusing on higher-level activities such as data modeling, business logic validation and ensuring that the final results meet business requirements. This demonstrates how AI agents can enhance modern data engineering workflows by combining automation with human supervision and knowledge.

3.0 Overview of the Data Pipeline

A data pipeline is a sequence of processes that move data from a source system to a destination where it can be analyzed and used for decision-making. Modern organizations collect large volumes of data from multiple sources, but raw data is often difficult to use directly because it may contain inconsistent naming conventions, duplicated information, missing values, or unsuitable formats.

To address these challenges, data engineers build pipelines that clean, transform, and organize data into a structure suitable for analysis. In this tutorial, an AI-assisted data pipeline was developed using Snowflake, dbt, and Claude AI. The pipeline follows a modern Extract-Load-Transform (ELT) architecture, where raw data is first loaded into a cloud data warehouse and transformed afterwards.

The objective of the pipeline is to transform raw customers and order data from Snowflake's sample dataset into a business-ready analytical table that summarizes sales performance. Claude AI was used throughout the development process to automate the generation of SQL models, configuration files, testing rules, documentation, and debugging activities.

Figure 3.1 illustrates the overall architecture of the AI-assisted data pipeline developed in this tutorial.

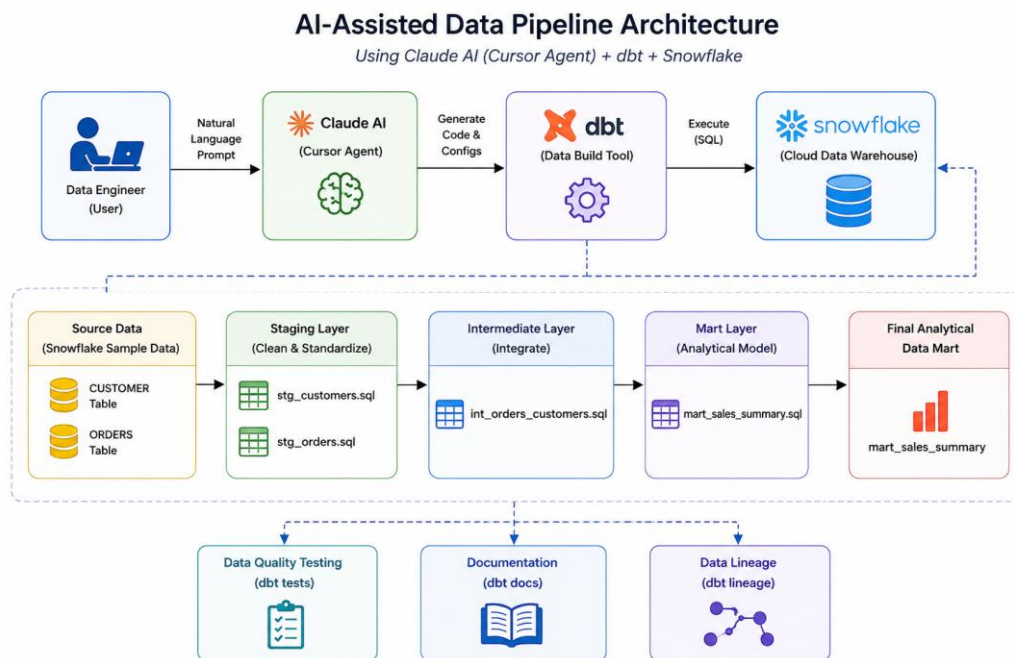


Figure 3.1: AI-Assisted Data Pipeline Architecture

The pipeline consists of the following major components:

1. Snowflake Sample Data (Data Source)
2. Snowflake Cloud Data Warehouse
3. dbt Transformation Framework
4. Claude AI through Cursor Agent
5. Final Analytical Data Mart
6. Documentation and Data Quality Testing

Each component plays an important role in transforming raw operational data into meaningful business information.

4.0 Pipeline Development and Implementation

This section explains the process of implementing the data engineering pipeline with AI assistance. The pipeline was built on top of Snowflake as the cloud data warehouse, dbt (Data Build Tool) as the transformation framework and Claude AI embedded in Cursor Agent as the development assistant. During the implementation, Claude was tasked with creating SQL models, YAML configuration files, documentation and debugging solutions from natural language prompts given by the users.

There are six key steps to the implementation process:

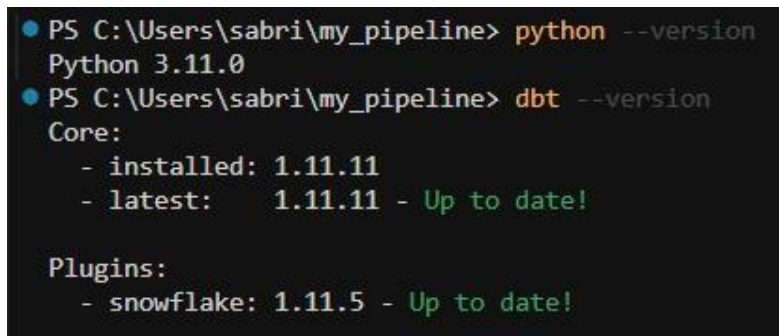
- Environment Setup and Verification
- Investigating the data from the sources
- AI pipeline development with Claude
- Project structure and model creation
- Pipeline Execution and Validation
- Analysis of results and Error Correction

4.1 Environment Setup and Verification

Before developing the data pipeline, it is necessary to make sure that all relevant software components are installed and configured appropriately. Three main technologies are used by the pipeline:

- Python
- Dbt (Data build tool)
- Snowflake

Python is necessary because dbt is implemented as a Python library. DBT may connect to the Snowflake cloud data warehouse via the Snowflake adapter. During pipeline development, configuration problems can be avoided by confirming the installation of these components.

A terminal window with a dark background and light blue text. It shows two commands being executed. The first command is 'python --version' which outputs 'Python 3.11.0'. The second command is 'dbt --version' which outputs 'Core:' followed by '- installed: 1.11.11' and '- latest: 1.11.11 - Up to date!'. Below that, it outputs 'Plugins:' followed by '- snowflake: 1.11.5 - Up to date!'.

```
PS C:\Users\sabri\my_pipeline> python --version
Python 3.11.0
PS C:\Users\sabri\my_pipeline> dbt --version
Core:
  - installed: 1.11.11
  - latest:    1.11.11 - Up to date!

Plugins:
  - snowflake: 1.11.5 - Up to date!
```

Figure 4.1.1: Python and dbt Version

The terminal output in Cursor following the execution of the commands ‘python –version’ and ‘dbt –version’ is displayed in Figure 4.1.1. The output verifies the successful installation of Python 3.11.0, dbt Core 1.11.11, and the Snowflake adaptor 1.11.5. The development environment is prepared for constructing the data pipeline because every component is designated as current.

After verifying the software environment, must also confirmed the access to the cloud data warehouse.

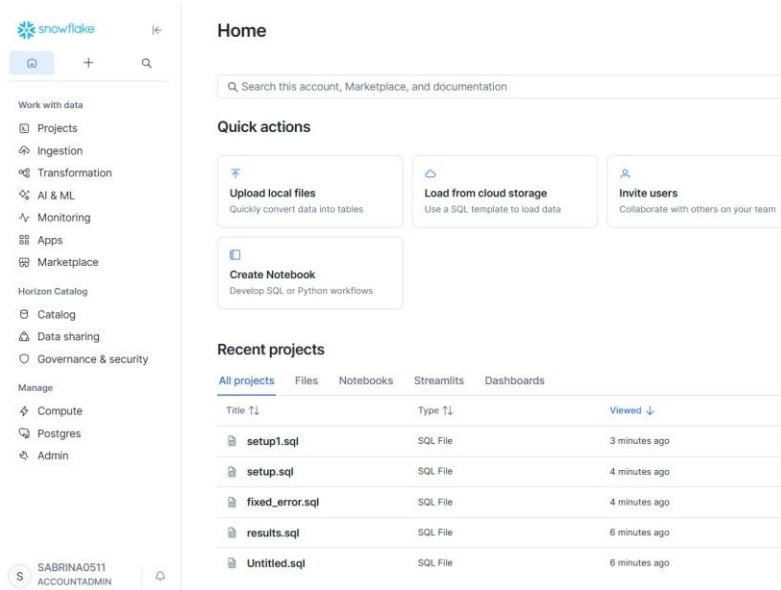


Figure 4.1.2: Snowflake Dashboard

The Snowflake home dashboard is shown in Figure 4.1.2 following a successful login with the ACCOUNTADMIN role. Snowflake serves as the central platform for storing both raw and transformed data throughout the pipeline. The dashboard attests to the account’s accessibility and activity. The Recent Projects area also displays a number of SQL files that show the work completed during this course, such as setup.sql, fixed_error.sql and results.sql.

4.2 Exploration of Source Data

It is important to know the structure and content of the source data before creating transformation models. The source data for this tutorial is from the TPC-H benchmark data from Snowflake. TPC-H is a popular sample dataset that represents realistic business transactions and customer data. The pipeline has included two sources tables which is ORDERS table and CUSTOMER table. These tables contain raw data needed to generate analytical sales summaries. The following is the raw ORDERS table:

My Workspace > setup.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small) Choose database Share ...

```

1  -- Create your database and schemas
2  CREATE DATABASE IF NOT EXISTS ANALYTICS;
3  CREATE SCHEMA IF NOT EXISTS ANALYTICS.STAGING;
4  CREATE SCHEMA IF NOT EXISTS ANALYTICS.MARTS;
5  | Ctrl+I to generate
6
7  -- Test that sample data exists
8  SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS LIMIT 5;

```

Results (4 hours ago)

Table Chart Pivot

5 rows 57ms

	# O_ORDERKEY	# O_CUSTKEY	A O_ORDERSTATUS	# O_TOTALPRICE	O_ORDERDATE	A O_ORDERPRICE	A O_CLERK
1	3000001	145618	F	30175.88	1992-12-17	4-NOT SPECIFIED	Clerk#00000001
2	3000002	1481	O	297999.63	1995-07-28	1-URGENT	Clerk#00000005
3	3000003	127432	O	345438.38	1997-11-04	5-LOW	Clerk#00000004
4	3000004	47423	O	135965.53	1996-06-13	4-NOT SPECIFIED	Clerk#00000000
5	3000005	84973	F	209937.09	1992-09-12	5-LOW	Clerk#00000000

Figure 4.2.1: Raw ORDERS Table

The raw ORDERS table from SNOWFLAKESAMPLE_DATA is shown in Figure 4.2.1. This table stores information about transactions involving customers. The important columns include:

- O_ORDERKEY
- O_CUSTKEY
- O_ORDERSTATUS
- O_TOTALPRICE
- O_ORDERDATE

The column names are the same as those in the source system, with the prefix O_ added to the beginning of each name. These names are technically correct, but not very user-friendly and may make analytical queries less readable. Thus, one of the goals of the staging layer is to normalize and simplify these column names.

The ORDERS table contains transactional information and is the foundation for business metrics like revenue, order volume and average order value.

My Workspace > setup1.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small) Choose database Share ...

```

1  -- Create your database and schemas
2  CREATE DATABASE IF NOT EXISTS ANALYTICS;
3  CREATE SCHEMA IF NOT EXISTS ANALYTICS.STAGING;
4  CREATE SCHEMA IF NOT EXISTS ANALYTICS.MARTS;
5
6
7  -- Test that sample data exists
8  SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER LIMIT 5;

```

Results (just now)

Table Chart Pivot 5 rows 2.0s

#	C_CUSTKEY	C_NAME	C_ADDRESS	C_NATIONKEY	C_PHONE	C_ACCTBAL	C_MKTSEGMENT
1	60001	Customer#000060001	9li4zQn9cX	14	24-678-784-9652	9957.56	HOUSEHOLD
2	60002	Customer#000060002	ThGBMJDwKzkoOxt	15	25-782-500-8435	742.46	BUILDING
3	60003	Customer#000060003	Ed hbPtXMTAsGGr	16	26-859-847-7640	2526.92	BUILDING
4	60004	Customer#000060004	NivCT2RVaavLyUnk	10	20-573-674-7999	7975.22	AUTOMOBILE
5	60005	Customer#000060005	1F3KM3ccEXEtI, BZ	12	22-741-208-1316	2504.74	MACHINERY

Figure 4.2.2: Raw CUSTOMER Table

The CUSTOMER table from SNOWFLAKESAMPLE_DATA is displayed in Figure 4.2.2. This table includes customer data such as customer identifiers, names, addresses, phone numbers, account balances and market segments. The important key includes:

- C_CUSTKEY
- C_NAME
- C_ADDRESS
- C_PHONE
- C_MKTSEGMENT

Like the ORDERS table, the CUSTOMER table has a prefix of C_ in its source-system naming conventions. These column names will be remapped during the staging process to make them easier to read and more consistent throughout the data pipeline.

The CUSTOMER and ORDERS tables are the basis of the analytical pipeline. The CUSTOMER table contains descriptive data for customers, and the ORDERS table contains data about what customers have bought.

4.3 AI-Assisted Pipeline Development Using Claude

This tutorial's main goal is to show how data engineering tasks can be assisted by artificial intelligence. A large portion of the development process was automated using Claude AI rather than having each SQL model and configuration file written by hand.

Cursor Agent, which enables the AI model to read project files, create new files, change configurations and help with debugging chores, was used to access Claude. This feature significantly reduces the quantity of manual coding needed.

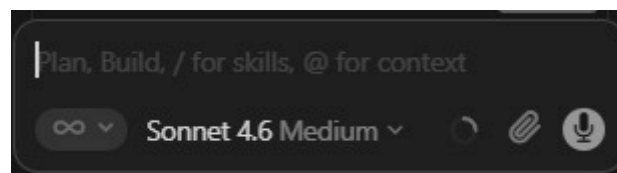


Figure 4.3.1: Claude AI Selected in Cursor Agent

The Cursor Agent interface with Claude Sonnet 4.6 Medium chosen as the AI model is displayed in Figure 4.3.1. Claude served as the primary AI assistant throughout the development process. Operating in Agent Mode was able to create files, modify code and arrange the project structure with little human labor using Claude. This capability enabled Claude to create and update multiple files across the dbt project automatically.

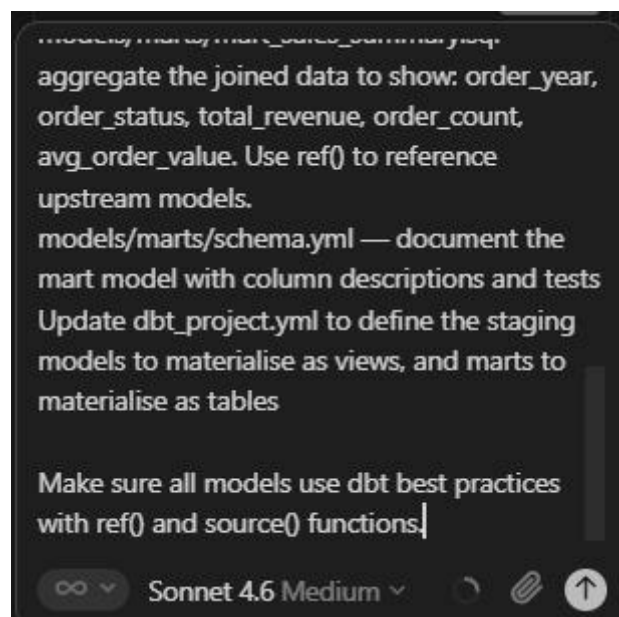


Figure 4.3.2: Initial Prompt for Pipeline Generation

In AI-assisted development, prompt engineering is crucial. The process of creating precise and comprehensive instructions that direct an AI model toward generating the expected outcome is known as prompt engineering.

The initial prompt sent to Claude is shown in Figure 4.3.2. The prompt asked Claude to produce:

- Staging models
- Intermediate models
- Mart models
- Schema files
- Dbt project configurations

The prompt also outlined a number of dbt best practices, such as using the ‘source()’ and ‘ref()’ procedures. The user ensured the created pipeline matched to suggested development standards by giving clear instructions.

Claude automatically constructed the entire pipeline structure by interpreting the prompt instead of manually generating each file.

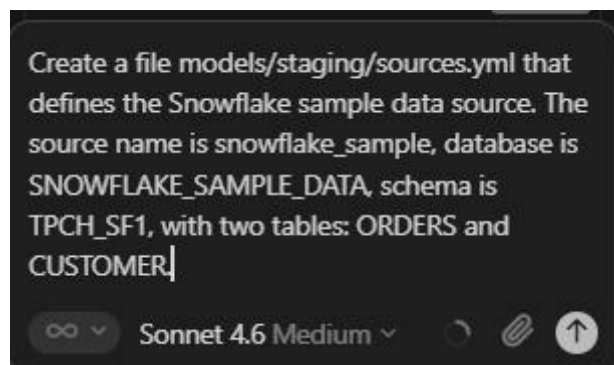


Figure 4.3.3: Prompt for Source Definition Creation

The ‘.yml’ sources document was created using a second prompt after the transformation models were generated. The raw source tables that dbt models will use are defined in this file. Figure 4.3.3 displays the definition of the prompt:

- Source name: snowflake_sample
- Database: SNOWFLAKE_SAMPLE_DATA
- Schema: TPCH_SF1
- Tables: CUSTOMER and ORDERS

Dbt can trace data lineage and preserve distinct connections between raw source data and modified models due to the created source definition.

AI may automate repetitive engineering activities while maintaining human developers' control over overall system design and business needs. This demonstrated by the usage of prompts throughout the development process.

4.4 Project Structure and Model Development

Once the source definitions were written, Claude wrote the full dbt project structure. Dbt's layered architecture is crucial for maintainability, readability and scalability, as it separates data transformations into staging, intermediate and mart layers. Every layer has a particular role and is part of the overall pipeline workflow. The complete dbt Project Structure is shown below:

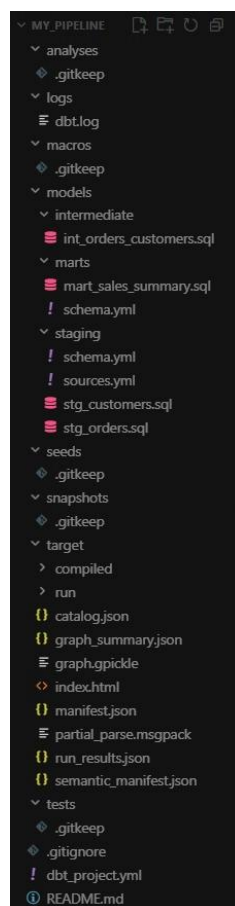


Figure 4.4.1: Complete Folder Structure

The full folder structure created by Claude in the dbt project is displayed in Figure 4.4.1. The project is designed in a layered architecture, following dbt's recommendations to group transformation models for their purpose.

The main folders contain:

- Staging layer: contains model that clean and standardize raw data
- Intermediate layer: includes models that integrate and enhance datasets
- Marts: contains analytical models used for reporting and business intelligence

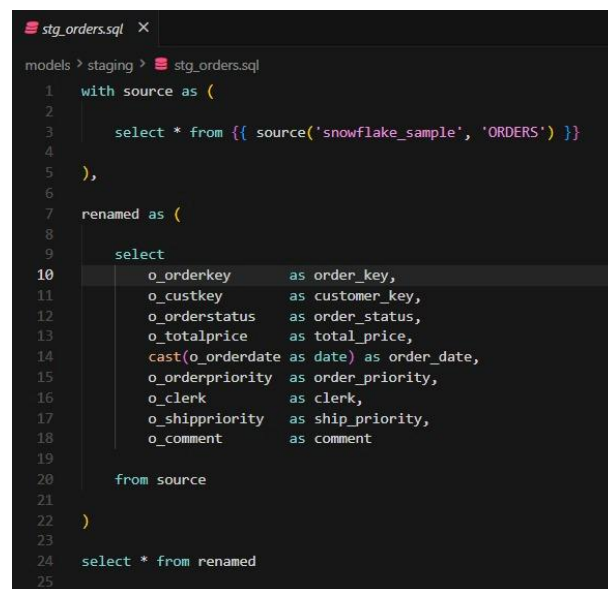
In addition to these folders, the project also contains configuration files such as:

- `dbt\_project.yml`
- `schema.yml`
- `sources.yml`

This will keep the pipeline organized and maintainable as the number of models grows.

4.4.1 Staging Layer Development

The staging layer is the first transformation layer in the dbt pipeline. The main purpose of this layer is to standardize the raw source data so that it can be readily utilized by the downstream models. For example, renaming columns, standardizing naming, convert data types and improving readability. The staging layer does not contain complex business logic. Rather, it concentrates on generating clean and consistent datasets.



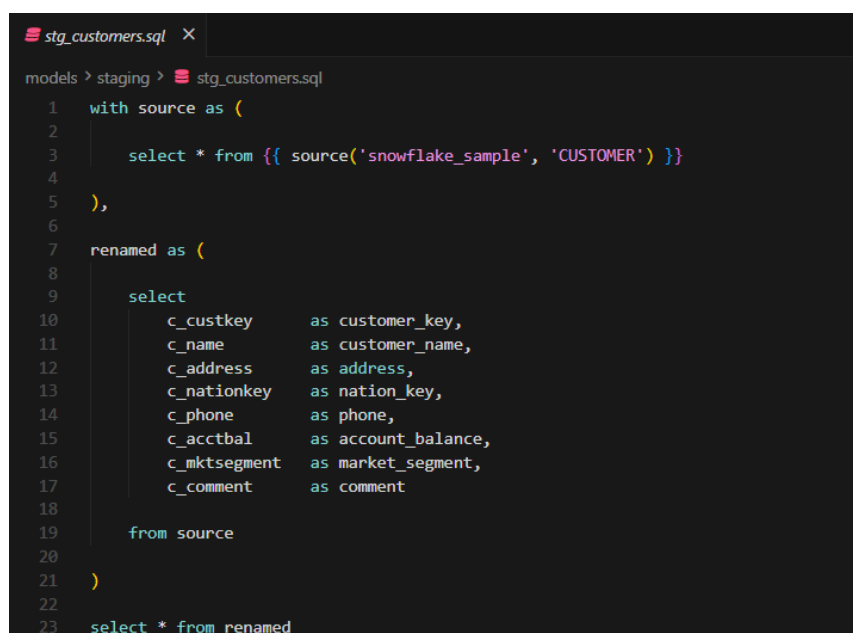
```
stg_orders.sql X
models > staging > stg_orders.sql
1  with source as (
2
3      select * from {{ source('snowflake_sample', 'ORDERS') }}
4
5  ),
6
7  renamed as (
8
9      select
10         o_orderkey      as order_key,
11         o_custkey       as customer_key,
12         o_orderstatus   as order_status,
13         o_totalprice    as total_price,
14         cast(o_orderdate as date) as order_date,
15         o_orderpriority as order_priority,
16         o_clerk         as clerk,
17         o_shippriority  as ship_priority,
18         o_comment       as comment
19
20     from source
21
22 )
23
24 select * from renamed
25
```

Figure 4.4.1.1: *stg_orders.sql*

Figure 4.4.1.1 shows the ‘stg_orders.sql’ model generated by Claude. This model uses the source() function to read data directly from the ORDERS source table. The model does several important transformations:

Raw Column	Transformed Column
O_ORDERKEY	order_key
O_CUSTKEY	customer_key
O_ORDERSTATUS	order_status
O_TOTALPRICE	total_price
O_ORDERDATE	order_date

An important change this model does is that it converts the order date to a standard DATE format. Standardizing data types helps ensure consistency across downstream models and analytical queries. Otherwise, the model also removes the O_ prefix from column names. This makes it easier to read and helps business users to better understand the data. The result of this model is a clean orders dataset that can be used for further transformations.



```

1  with source as (
2
3      select * from {{ source('snowflake_sample', 'CUSTOMER') }}
4
5  ),
6
7  renamed as (
8
9      select
10         c_custkey      as customer_key,
11         c_name         as customer_name,
12         c_address     as address,
13         c_nationkey    as nation_key,
14         c_phone        as phone,
15         c_acctbal      as account_balance,
16         c_mktsegment   as market_segment,
17         c_comment      as comment
18
19     from source
20
21 )
22
23 select * from renamed

```

Figure 4.4.1.2: stg_customers.sql

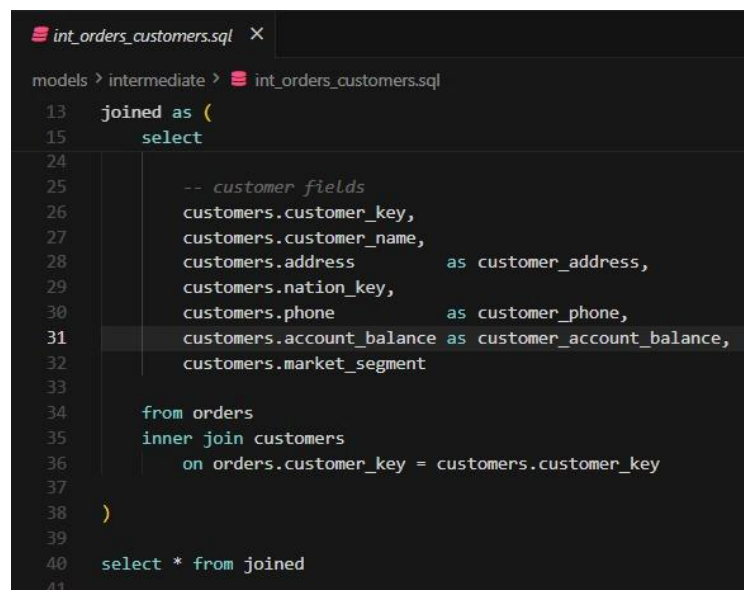
The Claude-generated model of the table stg_customers.sql is displayed in Figure 4.4.1.2. This model is like the orders staging model, except that it uses the source() function to read data from the CUSTOMER source table. The table below shows a few columns has been renamed:

Raw Column	Transformed Column
C_CUSTKEY	customer_key
C_NAME	customer_name
C_ADDRESS	address
C_PHONE	phone
C_MKTSEGMENT	market_segment

By removing the C_ prefix, the naming convention is more uniform throughout the project. The model also helps to make the customer information more readable, and it is standardized so that it can be easily merged with the order data in further transformations. Otherwise, this model produces a clean customer dataset with customer identifiers, contact information, account balances and market segment classifications.

4.4.2 Intermediate Layer Development

After the source data has been cleaned and standardized, the next step is to combine related datasets. The intermediate layer is responsible for integrating multiple datasets, applying business logic, creating enriched datasets and preparing data for analytical processing. This layer is a link between raw operational data and business-ready reporting data.



```

13  joined as (
14
15      select
16
17          -- customer fields
18          customers.customer_key,
19          customers.customer_name,
20          customers.address      as customer_address,
21          customers.nation_key,
22          customers.phone        as customer_phone,
23          customers.account_balance as customer_account_balance,
24          customers.market_segment
25
26      from orders
27      inner join customers
28          on orders.customer_key = customers.customer_key
29
30  )
31
32  select * from joined
33
34
35
36
37
38
39
40
41

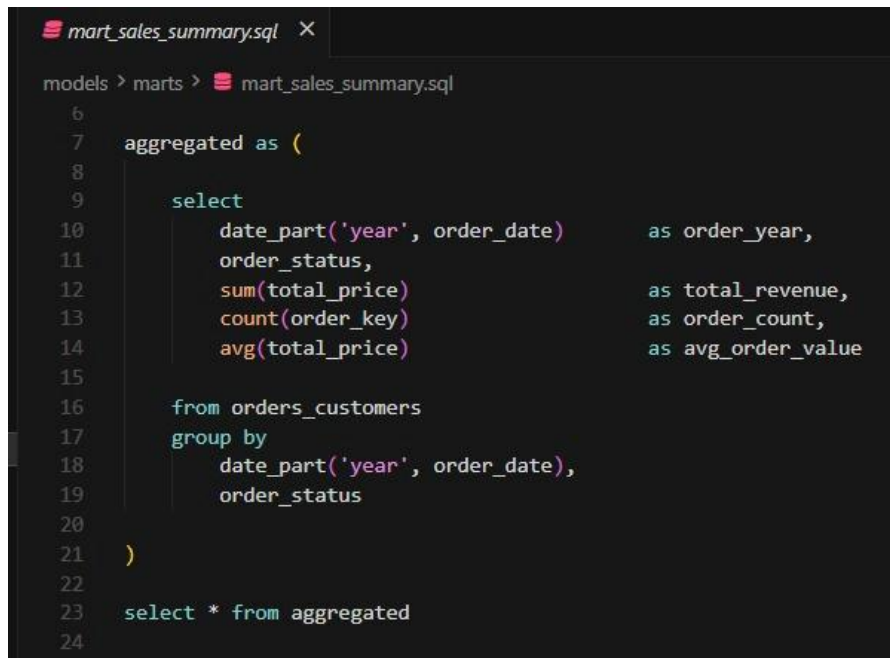
```

Figure 4.4.2.1: *int_orders_customers.sql*

The model for the `int_orders_customers.sql` is found in Figure 4.4.2.1. This model is a combination of the outputs of the staging models, with the information from the customers and orders models being joined together. The join is done on the `customer_key` field which is present in both datasets.

4.4.3 Mart Layer Development

The mart layer is the last layer of transformation. The purpose of data marts is to provide support for reporting, dashboards, and business decision making.



```
mart_sales_summary.sql X
models > marts > mart_sales_summary.sql
6
7 aggregated as (
8
9     select
10         date_part('year', order_date) as order_year,
11         order_status,
12         sum(total_price) as total_revenue,
13         count(order_key) as order_count,
14         avg(total_price) as avg_order_value
15
16     from orders_customers
17     group by
18         date_part('year', order_date),
19         order_status
20
21 )
22
23 select * from aggregated
24
```

Figure 4.4.3.1: mart_sales_summary.sql

The model is shown in Figure 4.4.3.1, and it combines integrated customer-order data to create a sales summary table. The model groups records are arranged by Order year, Order status and some calculation like total revenue, total number of orders and average order value. These calculations convert millions of detailed order records into an analytical dataset that can be used for business reporting. The mart layer is the end result of the transformation pipeline and is the main data source for analysis.

4.4.4 Data Quality and Documentation Configuration

In today's data engineering, developing transformation models is just a component of the process. Ensuring data quality and documentation is also crucial for reliability and maintainability. In this tutorial, Claude automatically generated both documentation files and testing configurations.

```

! schema.yml ×
models > marts > ! schema.yml
1  version: 2
2
3  models:
4    - name: mart_sales_summary
5      description: >
6        Sales summary mart aggregating order data by year and status.
7        Materialised as a table for fast downstream querying.
8        Joins orders and customer data via the intermediate layer.
9      columns:
10     - name: order_year
11       description: Calendar year the orders were placed, extracted from order_date.
12       tests:
13         - not_null
14     - name: order_status
15       description: >
16         Status of the orders in this group.
17         F = Fulfilled, O = Open, P = Pending.
18       tests:
19         - not_null
20         - accepted_values:
21             arguments:
22               values: ['F', 'O', 'P']
23     - name: total_revenue
24       description: Sum of total_price for all orders in this year/status group.
25       tests:
26         - not_null
27     - name: order_count
28       description: Number of orders in this year/status group.
29       tests:
30         - not_null
31     - name: avg_order_value
32       description: Average total_price per order within this year/status group.
33       tests:
34         - not_null
35

```

Figure 4.4.4.1: schema.yml for Marts

Figure above show the mart layer's schema.yml file. This file is used for documentation which includes descriptions that help future developers and analysis understand the meaning and purpose of each field. Additionally, it defines the data quality checks like 'accepted_values' and 'not_null', which guarantee that the data complies with predetermined guidelines each time the dbt test is run.

```

! sources.yml x
models > staging > ! sources.yml
1  version: 2
2
3  sources:
4    - name: snowflake_sample
5      database: SNOWFLAKE_SAMPLE_DATA
6      schema: TPC_SF1
7      description: >
8        TPC-H benchmark dataset provided by Snowflake as a free sample database.
9        Scale factor 1 (~1 GB of raw data).
10
11  tables:
12    - name: ORDERS
13      description: One row per order placed in the system.
14      columns:
15        - name: O_ORDERKEY
16          description: Primary key for the orders table.
17        - name: O_CUSTKEY
18          description: Foreign key to the customer who placed the order.
19        - name: O_ORDERSTATUS
20          description: "Status of the order: F (Fulfilled), O (Open), or P (Pending)."

```

Figure 4.4.4.2: *sources.yml*

The `sources.yml` file used in the staging layer is shown in Figure 14. This file documents the `ORDERS` and `CUSTOMER` tables and registers the Snowflake example dataset as a source in dbt. In order to enhance data documentation and lineage tracking, it additionally provides explanations for significant columns.

4.5 Pipeline Execution, Testing and Documentation

The next step was to run the pipeline and confirm that the transformations were successful once all models and configuration files had been created. Developers can create models, verify the integrity of data and produce project documentation using a number of commands offered by dbt.

The three most crucial commands are:

- dbt run
- dbt test
- dbt docs generate

These commands ensure that the pipeline not only executes successfully but also produces reliable and well-documented outputs.

Pipeline Execution Using dbt Run

All transformation models specified inside the project are executed using the ‘dbt run’ command. Based on model dependencies created by the ‘ref()’ function, dbt automatically decides the proper order of execution during execution. Upstream models are developed before downstream models that rely on them due to this dependency management.

```
PS C:\Users\sabri\my_pipeline> dbt run
09:56:24 Running with dbt=1.11.11
09:56:25 Registered adapter: snowflake=1.11.5
09:56:26 Found 4 models, 17 data tests, 2 sources, 536 macros
09:56:26
09:56:26 Concurrency: 1 threads (target='dev')
09:56:26
09:56:28 1 of 4 START sql view model DEV.stg_customers ..... [RUN]
09:56:29 1 of 4 OK created sql view model DEV.stg_customers ..... [SUCCESS 1 in 0.75s]
09:56:29 2 of 4 START sql view model DEV.stg_orders ..... [RUN]
09:56:29 2 of 4 OK created sql view model DEV.stg_orders ..... [SUCCESS 1 in 0.22s]
09:56:29 3 of 4 START sql view model DEV.int_orders_customers ..... [RUN]
09:56:29 3 of 4 OK created sql view model DEV.int_orders_customers ..... [SUCCESS 1 in 0.44s]
09:56:29 4 of 4 START sql table model DEV.mart_sales_summary ..... [RUN]
09:56:30 4 of 4 OK created sql table model DEV.mart_sales_summary ..... [SUCCESS 9 in 0.79s]
09:56:30
09:56:30 Finished running 1 table model, 3 view models in 0 hours 0 minutes and 3.99 seconds (3.99s).
09:56:30 Completed successfully
09:56:30 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=4
```

Figure 4.5.1: Successful dbt Run

The terminal output following the execution of the ‘dbt run’ command is displayed in Figure 4.5.1. The output indicates that all four models were successfully materialized as shown in Table 4.5:

Table 4.5.1 Models Materialization Type

Model	Materialisation Type
stg_customers	View
stg_orders	View
int_orders_customers	View
mart_sales_summary	Table

As shown in Figure 4.5.1, the execution process completed successfully with:

- PASS = 4
- WARN = 0
- ERROR = 0

This result confirms that the SQL syntax generated by Claude was valid. The Snowflake connection was functioning correctly. The dependencies between models were configured properly. All transformation logic executed without failure.

The fact that distinct materialization techniques were applied to various levels is a crucial finding. Since the staging and intermediate models mostly function as transformation steps and don't need to be permanently stored, they were materialized as views. Views guarantee that transformations always reflect the most recent source data while lowering storage requirements. The mart model is a business-facing analytical dataset that is anticipated to be regularly accessed, therefore it was materialized as a table. Query performance and reporting efficiency are enhanced when the final result is stored as a table. The DBT run's successful completion attests to the AI-generated pipeline's ability to convert raw source data into analytical outputs without the need for human involvement.

Data Quality Validation Using dbt Test

If the quality of the final data cannot be assured, building a data pipeline is insufficient. Therefore, a crucial part of current data engineering is data quality validation. Developers can specify validation rules directly in schema files due to dbt's integrated testing features. These tests automatically confirm that the modified data meets predetermined standards for quality. Three types of data quality tests are used in the tutorial as shown in Table 4.5.2:

Table 4.5.2 Data Quality Test

Quality test	Description	Example
Not_null Test	Verifies a column does not contain missing values. Missing values in the field could lead to incorrect reporting and analytical results	- customer_key should never be null. - order_key should never be null. - total_revenue should never be null.
Unique Test	Verifies that every value in a column is unique. Duplicate values could indicate data integrity problems	- order_key should uniquely identify each order. - customer_key should uniquely identify each customer.
Accepted_values Test	Ensures that a column only contains predefined valid values. Any unexpected value would indicate a data quality issue.	For example, the order status field may only contain: - F (Fulfilled) - O (Open) - P (Pending)

```

PS C:\Users\sabri\my_pipeline> dbt test
09:56:37 Running with dbt=1.11.11
09:56:38 Registered adapter: snowflake=1.11.5
09:56:39 Found 4 models, 17 data tests, 2 sources, 536 macros
09:56:39
09:56:39 Concurrency: 1 threads (target='dev')
09:56:39
09:56:40 1 of 17 START test accepted_values_mart_sales_summary_order_status_F_O_P .... [RUN]
09:56:41 1 of 17 PASS accepted_values_mart_sales_summary_order_status_F_O_P ..... [PASS in 0.98s]
09:56:41 2 of 17 START test accepted_values_stg_orders_order_status_F_O_P ..... [RUN]
09:56:42 2 of 17 PASS accepted_values_stg_orders_order_status_F_O_P ..... [PASS in 1.00s]
09:56:42 3 of 17 START test not_null_mart_sales_summary_avg_order_value ..... [RUN]
09:56:42 3 of 17 PASS not_null_mart_sales_summary_avg_order_value ..... [PASS in 0.12s]
09:56:42 4 of 17 START test not_null_mart_sales_summary_order_count ..... [RUN]
09:56:43 4 of 17 PASS not_null_mart_sales_summary_order_count ..... [PASS in 0.12s]
09:56:43 5 of 17 START test not_null_mart_sales_summary_order_status ..... [RUN]
09:56:43 5 of 17 PASS not_null_mart_sales_summary_order_status ..... [PASS in 0.18s]
09:56:43 6 of 17 START test not_null_mart_sales_summary_order_year ..... [RUN]
09:56:43 6 of 17 PASS not_null_mart_sales_summary_order_year ..... [PASS in 0.11s]
09:56:43 7 of 17 START test not_null_mart_sales_summary_total_revenue ..... [RUN]
09:56:43 7 of 17 PASS not_null_mart_sales_summary_total_revenue ..... [PASS in 0.11s]
09:56:43 8 of 17 START test not_null_stg_customers_customer_key ..... [RUN]
09:56:44 8 of 17 PASS not_null_stg_customers_customer_key ..... [PASS in 1.43s]
09:56:44 9 of 17 START test not_null_stg_customers_customer_name ..... [RUN]
09:56:46 9 of 17 PASS not_null_stg_customers_customer_name ..... [PASS in 1.51s]

```



```

09:56:44 9 of 17 START test not_null_stg_customers_customer_name ..... [RUN]
09:56:46 9 of 17 PASS not_null_stg_customers_customer_name ..... [PASS in 1.51s]
09:56:46 10 of 17 START test not_null_stg_customers_nation_key ..... [RUN]
09:56:46 10 of 17 PASS not_null_stg_customers_nation_key ..... [PASS in 0.40s]
09:56:46 11 of 17 START test not_null_stg_orders_customer_key ..... [RUN]
09:56:47 11 of 17 PASS not_null_stg_orders_customer_key ..... [PASS in 0.49s]
09:56:47 12 of 17 START test not_null_stg_orders_order_date ..... [RUN]
09:56:48 12 of 17 PASS not_null_stg_orders_order_date ..... [PASS in 0.91s]
09:56:48 13 of 17 START test not_null_stg_orders_order_key ..... [RUN]
09:56:48 13 of 17 PASS not_null_stg_orders_order_key ..... [PASS in 0.41s]
09:56:48 14 of 17 START test not_null_stg_orders_order_status ..... [RUN]
09:56:48 14 of 17 PASS not_null_stg_orders_order_status ..... [PASS in 0.25s]
09:56:48 15 of 17 START test not_null_stg_orders_total_price ..... [RUN]
09:56:49 15 of 17 PASS not_null_stg_orders_total_price ..... [PASS in 0.26s]
09:56:49 16 of 17 START test unique_stg_customers_customer_key ..... [RUN]
09:56:49 16 of 17 PASS unique_stg_customers_customer_key ..... [PASS in 0.43s]
09:56:49 17 of 17 START test unique_stg_orders_order_key ..... [RUN]
09:56:50 17 of 17 PASS unique_stg_orders_order_key ..... [PASS in 0.63s]
09:56:50
09:56:50 Finished running 17 data tests in 0 hours 0 minutes and 10.53 seconds (10.53s).
09:56:50
09:56:50 Completed successfully
09:56:50
09:56:50 Done. PASS=17 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=17

```

Figure 4.5.2 & 4.5.3: dbt Test Results

Figure 4.5.2 and 4.5.3 show the results produced after executing the ‘dbt test’ command. A total of 17 tests were executed across the project. The final result shows 17 PASS, 0 WARN and 0 ERROR.

The modified datasets meet all specified quality restrictions, as confirmed by the successful completion of all tests. This result is very significant since it shows that the pipeline not only runs smoothly but also generates accurate and dependable data. Errors like missing entries, duplicate keys or invalid values could go unnoticed and have a detrimental effect on business reporting if testing isn’t done.

Documentation Generation

Documentation becomes more crucial as data projects grow in size and complexity. The purpose of datasets, transformations and data lineage linkages can be better understood by developers, analysts and future project participants with the support of documentation.

By using the ‘dbt docs generate’ command, dbt offers automatic documentation generation. This command retrieves data from Models, Sources, Schema files and Tests. This will create a website for interactive documentation.

```

PS C:\Users\sabri\my_pipeline> dbt docs generate
>> dbt docs serve
09:57:05 Running with dbt=1.11.11
09:57:06 Registered adapter: snowflake=1.11.5
09:57:07 Found 4 models, 17 data tests, 2 sources, 536 macros
09:57:07
09:57:07 Concurrency: 1 threads (target='dev')
127.0.0.1 - - [11/Jun/2026 17:57:21] "GET /catalog.json?cb=1781171841120 HTTP/1.1" 200 -
127.0.0.1 - - [11/Jun/2026 17:57:21] code 404, message File not found
127.0.0.1 - - [11/Jun/2026 17:57:21] "GET /%7B%7B%20getIcon(item.type,%20'on')%20%7D%7D HTTP/1.1" 404 -
127.0.0.1 - - [11/Jun/2026 17:57:21] code 404, message File not found
127.0.0.1 - - [11/Jun/2026 17:57:21] "GET /%7B%7B%20getIcon(item.type,%20'off')%20%7D%7D HTTP/1.1" 404 -
127.0.0.1 - - [11/Jun/2026 17:57:22] code 404, message File not found
127.0.0.1 - - [11/Jun/2026 17:57:22] "GET /%7Brequire('./assets/favicons/favicon.ico')%7D HTTP/1.1" 404 -

```

Figure 4.5.4: dbt Documentation Generation

The terminal output following the execution of the commands is displayed in Figure 4.5.4:

‘dbt docs generate’

‘dbt docs serve’

The process of documenting identified 4 models, 2 source tables and 17 tests. Automated documentation enhances teamwork and lessens the amount of manual labor needed to maintain project knowledge.

Data Lineage Visualization

The ability to automatically create data lineage diagram is one of dbt’s most potent features. The capability to track data as it passes through various pipeline stages is known as data lineage. Engineers can better understand where information comes from, which models rely on one another and how modifications impact system downstream. Data lineage is especially crucial for impact analysis, governance and debugging.



Figure 4.5.5: Lineage Graph

Figure 4.5.5 presents the lineage graph generated by dbt. The entire data flow via the pipeline is shown in the pipeline. It includes CUSTOMER and ORDER source table, stg_customers, stg_orders, int_orders_customers and mart_sales_summary model. The lineage graph shows how raw source data is gradually converted into an analytical dataset that is ready for business use. Developers do not need to manually create dependency diagrams since dbt uses 'ref()' and 'source()' relationship to automatically provide lineage information. This feature makes project maintenance easier and increases transparency.

Project Documentation Website

For each source and model in the project, dbt creates comprehensive metadata pages in addition to the lineage graph. These pages include details like:

- Table descriptions.
- Column descriptions.
- Data quality tests.
- Source metadata.
- Dependency relationships.

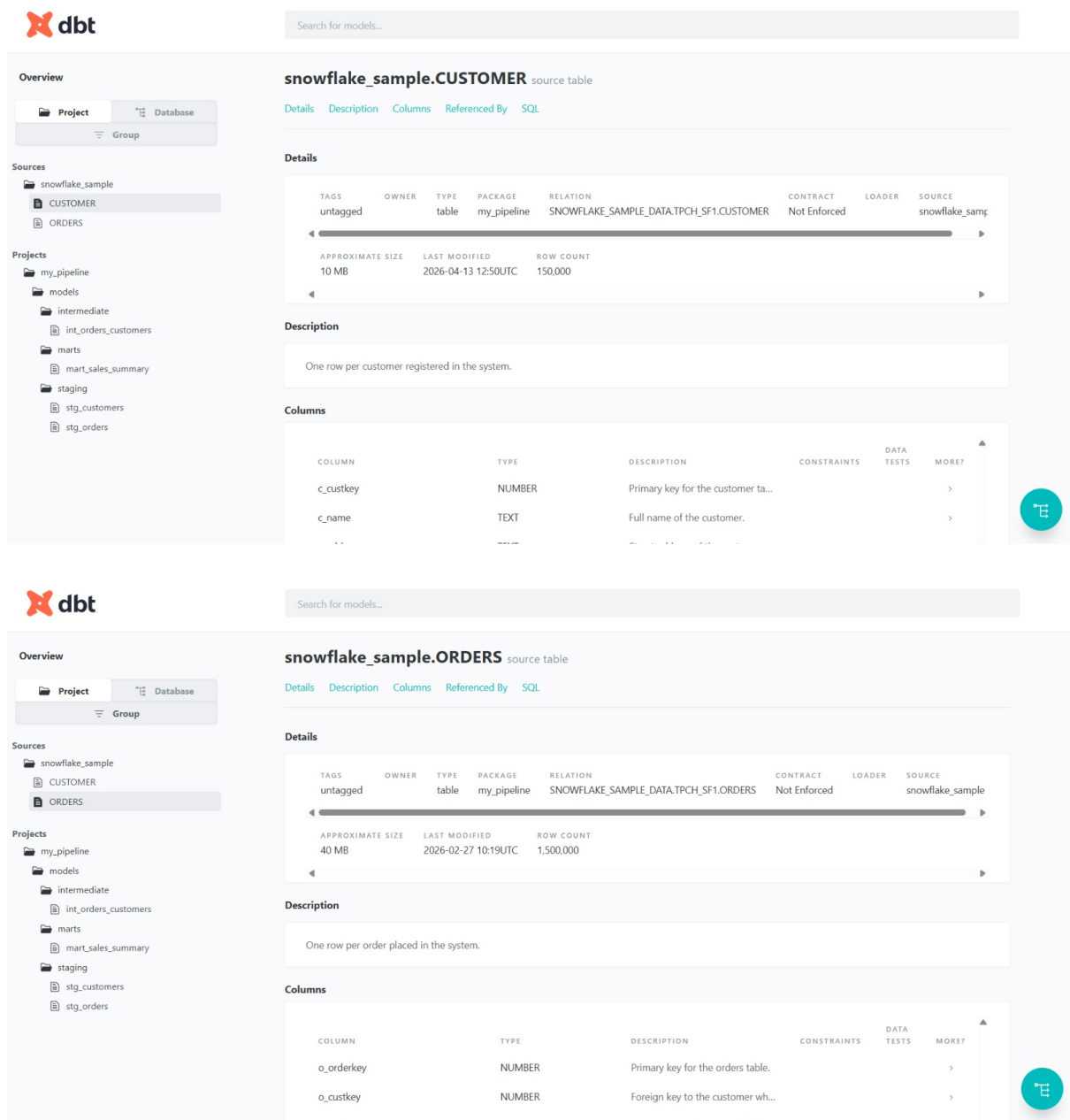


Figure 4.5.6 & 4.5.7: dbt Documentation

Examples of the documentation pages produced by dbt are displayed in Figure 4.5.6 and 4.5.7. For the CUSTOMER and ORDERS source tables, the documentation pages include comprehensive details such as:

- Source definitions.
- Column descriptions.
- Metadata information.
- Data lineage relationships.

Additionally, the documentation enhances comprehension of how individual datasets contribute to the final analytical output and offers visibility into the pipeline’s structure. Using the project files as a guide, dbt automatically generates and updates documentation. As a result, the pipeline is easy to manage and the documentation is kept accurate.

4.6 Results in Snowflake

The final output saved in Snowflake following pipeline execution is shown in Figure 4.6. Aggregated sales data from 1992 to 1998, including metrics like total orders, total revenue, and average order value are included in the table. This shows that the pipeline was successful in turning about 1.5 million raw order records into a clear, useful summary table that could be used for reporting and analysis.

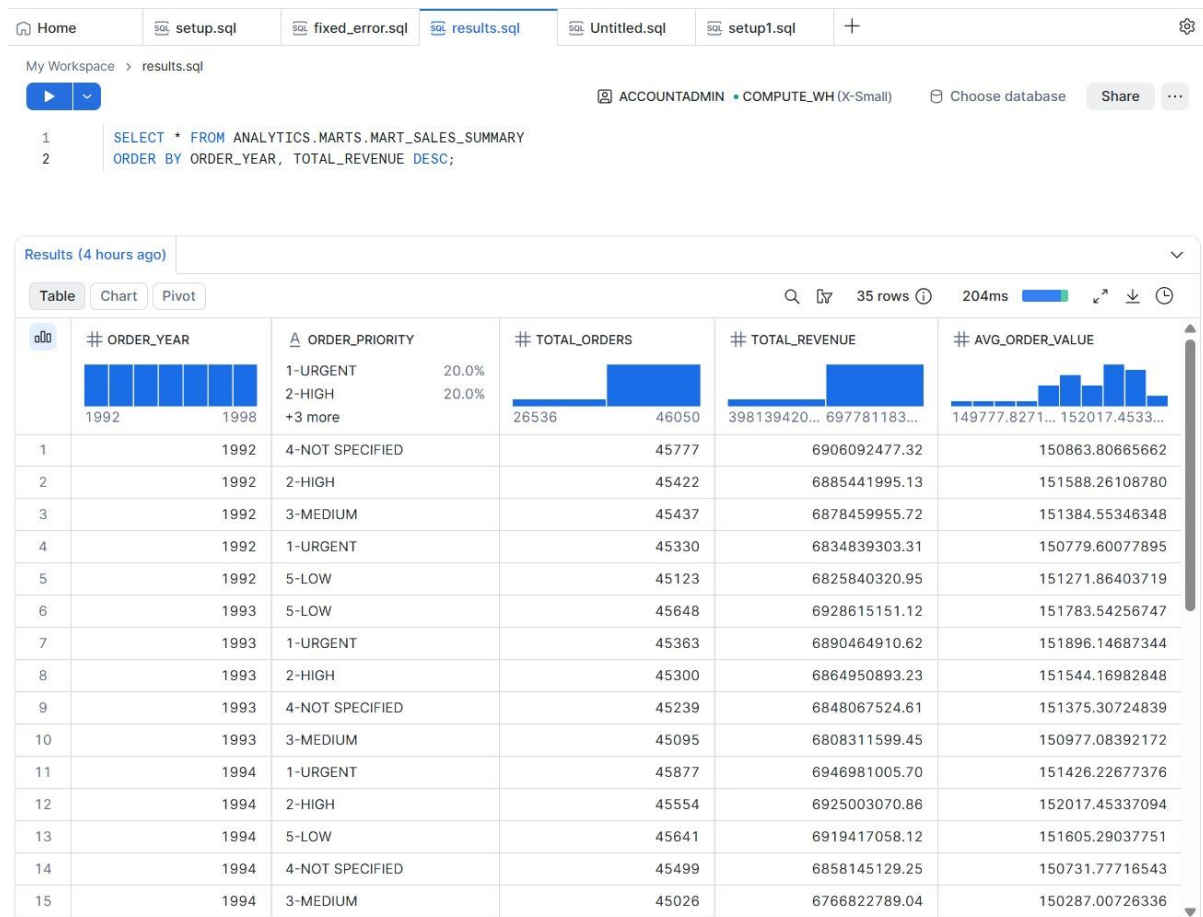


Figure 4.6: Final Mart Table

4.7 Error Detection and Correction

An earlier unsuccessful DBT test run during the pipeline's development phase is seen in Figure 4.7.1. The pipeline configuration was not yet quite accurate, as all 17 tests in this try returned an ERROR result. There were two primary causes of the failures. First, dbt was unable to find the necessary objects during testing since the 'ANALYTICS' database was not yet created in Snowflake. The second is the schema's 'accepted_values' test. The syntax used in the yml file was outdated and incompatible with DBT version 1.11. Consequently, it was unable to effectively complete any of the data quality checks. This situation illustrates a typical problem in data engineering projects, where pipeline execution may be impacted by configuration and compatibility problems.

```
PS C:\Users\sabri\my_pipeline> dbt test
09:53:04 Running with dbt=1.11.11
09:53:05 Registered adapter: snowflake=1.11.5
09:53:06 Found 4 models, 17 data tests, 2 sources, 536 macros
09:53:06
09:53:06 Concurrency: 1 threads (target='dev')
09:53:06
09:53:07 1 of 17 START test accepted_values_mart_sales_summary_order_status_F_O_P .... [RUN]
09:53:07 1 of 17 ERROR accepted_values_mart_sales_summary_order_status_F_O_P ..... [ERROR in 0.61s]
09:53:07 2 of 17 START test accepted_values_stg_orders_order_status_F_O_P ..... [RUN]
09:53:07 2 of 17 ERROR accepted_values_stg_orders_order_status_F_O_P ..... [ERROR in 0.25s]
09:53:07 3 of 17 START test not_null_mart_sales_summary_avg_order_value ..... [RUN]
09:53:08 3 of 17 ERROR not_null_mart_sales_summary_avg_order_value ..... [ERROR in 0.10s]
09:53:08 4 of 17 START test not_null_mart_sales_summary_order_count ..... [RUN]
09:53:08 4 of 17 ERROR not_null_mart_sales_summary_order_count ..... [ERROR in 0.09s]
09:53:08 5 of 17 START test not_null_mart_sales_summary_order_status ..... [RUN]
09:53:08 5 of 17 ERROR not_null_mart_sales_summary_order_status ..... [ERROR in 0.10s]
09:53:08 6 of 17 START test not_null_mart_sales_summary_order_year ..... [RUN]
09:53:08 6 of 17 ERROR not_null_mart_sales_summary_order_year ..... [ERROR in 0.10s]
09:53:08 7 of 17 START test not_null_mart_sales_summary_total_revenue ..... [RUN]
09:53:08 7 of 17 ERROR not_null_mart_sales_summary_total_revenue ..... [ERROR in 0.08s]
09:53:08 8 of 17 START test not_null_stg_customers_customer_key ..... [RUN]
09:53:08 8 of 17 ERROR not_null_stg_customers_customer_key ..... [ERROR in 0.11s]
```

Figure 4.7.1: dbt Test Errors

The Cursor Agent's reaction to the error signals is displayed in Figure 4.7.2. Claude was able to recognize the underlying issues and recommend the appropriate solutions. It automatically fixed the schema test syntax and updated the project setup. This example shows how developers can troubleshoot problems more effectively and spend less time looking for solutions by using AI-assisted tools.

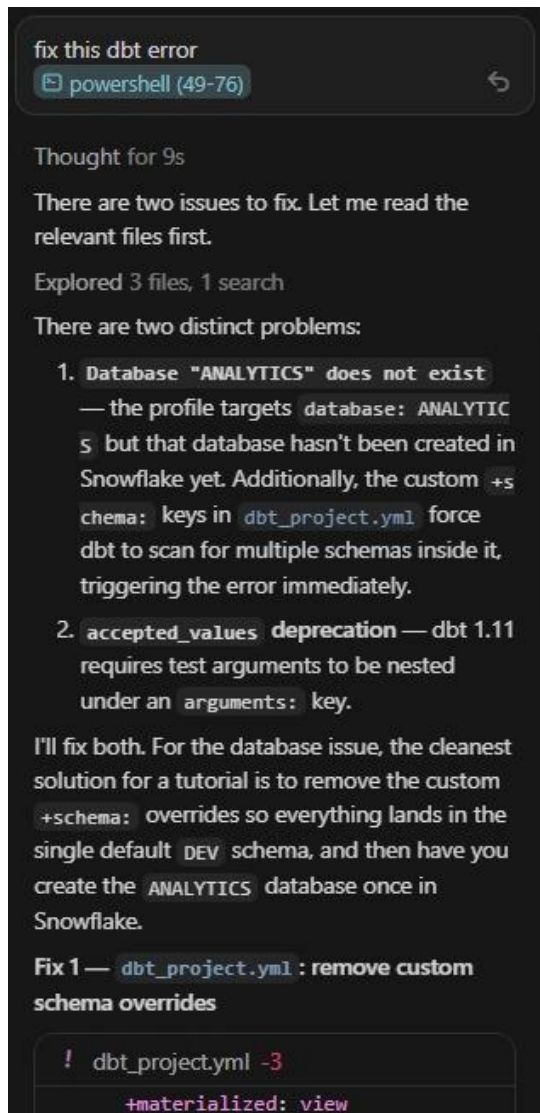


Figure 4.7.2: Cursor Agent Fixing the Error

5.0 Use Case Diagram



Figure 5.1 Use Case Diagram of the AI-Assisted Data Engineering Pipeline

Figure 5.1 is the use case diagram for the AI-Assisted Data Engineering Pipeline created in this tutorial. The Data Engineer is the primary actor and is responsible for the development environment, exploring the source data, sending prompt to Claude AI, running the pipeline, running data quality tests, creating documentation, viewing data lineage and querying the final results.

There are multiple supporting systems that work with Data Engineer during the pipeline development process. When errors in the pipeline arise, Claude AI can help generate dbt models, write YAML pipeline configuration files and fix the errors. The dbt framework is what runs the transformations, checks data quality and creates documentation of the project. Snowflake serves as the cloud data warehouse, delivering the raw source data to the pipeline and storing the completed analytical tables generated by the pipeline.

Also, some use cases are related to each other as represented in the diagram shown. Transform Data is a use case that is also included in the Execution Pipeline use case because during pipeline execution, the data needs to be transformed. Likewise, View Data Lineage is included in Generate Documentation because lineage information is automatically generated when you write documentation for dbt. In general, the use case diagram shows the overall flow of the Data Engineer, Claude AI, dbt, and Snowflake working together to develop, test, document, and upkeep the AI-powered data engineering pipeline.

6.0 Reflection

Gui Kah Sin

This tutorial provided valuable insights into AI-assisted data engineering by showing how an end-to-end data pipeline can be developed with the help of AI agents. The tutorial introduced several new technologies, including Snowflake, dbt, and YAML (YML) configuration files, which play important roles in modern data engineering workflows. Through the use of Snowflake sample data as the data source, a better understanding was gained of how data moves through different transformation layers within a dbt project. The staging layer focused on data cleaning, standardisation and column renaming, while the intermediate layer integrated data from multiple tables through joins to create enriched datasets. The mart layer then performed aggregations to produce business-ready analytical outputs. In addition, the tutorial also introduced the concept of materialisation in dbt where staging and intermediate models were configured as views while mart models were configured as tables. This provided a better

understanding of how different models can be stored in Snowflake based on their purpose within the data pipeline. Overall, the tutorial enhanced understanding of modern data engineering architecture, the practical use of cloud data warehousing and the growing role of AI in automating development tasks while still requiring human expertise for validation, data modelling and business logic design.

Sabrina Heng Wei Qi

This tutorial provided valuable exposure to modern data engineering practices, particularly using dbt (Data Build Tool) and Snowflake for building an ELT pipeline. While prior knowledge of SQL, databases, and data pipelines provided a foundation, the tutorial highlighted the importance of organizing transformations into staging, intermediate, and mart layers to improve maintainability, traceability, and debugging efficiency. A key learning outcome was understanding that dbt functions as an orchestration and transformation management tool, while the actual data processing is executed within Snowflake. The tutorial also reinforced the distinction between traditional ETL and modern ELT architectures. Additionally, the use of AI tools such as Claude through Cursor Pro accelerated development and troubleshooting by generating dbt models and resolving configuration issues efficiently. However, the experience demonstrated that AI-generated outputs still require domain knowledge, accurate prompts, and human validation to ensure correctness. Future enhancements could include automated scheduling, incremental data loading, more comprehensive data quality testing, and improved prompt engineering to support a more robust and production-ready data pipeline

Woo Cheng Shuan

The use of AI prompting to create a whole data pipeline was demonstrated in this tutorial. Based on specified requirements, AI technologies like Cursor Agent and Claude were able to effectively produce dbt models, configuration files and project structures. Understanding how data engineering procedures like ingestion, transformation and load are implemented in dbt through the staging, intermediate and mart layers was a crucial learning objective. Seeing these ideas put into practice made it easier to relate academic understanding to practical application. The need of clear prompts was also emphasized in the course, since the quality of the output

produced depended on the instructions given. AI helped with code generation and debugging, but human comprehension was still necessary to verify outcomes and guarantee accuracy. Overall, the event showed how AI may enhance current data engineering operations and increase efficiency.

7.0 Reference

Anthropic. (2024). *The Claude 3 model family: Opus, Sonnet, Haiku* [Model card]. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf

Cursor Docs — Agent, Rules, MCP, Skills & CLI. (2026). Cursor Documentation. <https://docs.cursor.com>

Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach. In *Pearson.com*. Pearson. <https://www.pearson.com/en-us/subject-catalog/p/artificial-intelligence-a-modern-approach/P2000000003500/9780137505135?srltid=AfmBOoq3cKEdScuCYNTWl6NNP7CdbQ-Q27PeezA92BYv07azlzwJqca->

Sikka, B., Dr. Priyender, & Dr. Prashant. (2023). *Practical Data Analytics for BFSI: Leveraging Data Science for Driving Decisions in Banking, Financial Services, and Insurance Operations*. Orange Education Pvt Limited.